

Autonomous Vehicle Steering Control Using Behavioral Cloning and Deep Convolutional Neural Networks

Course-CSE445 (Section-2)

Project Group No: 03

Al Murtuza Arabi (2212074642)

Sakib Newaz Chowdhury (1921291642)

Abstract—Autonomous driving systems have become one of the most important research areas in artificial intelligence, robotics, and computer vision. This project presents an end-to-end behavioral cloning system capable of learning steering behavior directly from driving images using deep learning techniques. A Convolutional Neural Network (CNN) inspired by the NVIDIA autonomous driving architecture is implemented using TensorFlow and Keras frameworks. The model is trained using the Udacity Self-Driving Car Simulator dataset containing multi-camera images and steering labels. Various preprocessing and augmentation techniques including image cropping, resizing, YUV color conversion, random flipping, random translation, and brightness adjustment are applied to improve generalization and reduce overfitting. The trained model is deployed in a real-time simulation environment using SocketIO communication. Experimental results demonstrate stable lane-following capability, smooth steering behavior, and successful autonomous navigation under simulator conditions.

Index Terms—Behavioral Cloning, Autonomous Driving, Deep Learning, CNN, NVIDIA Architecture, Computer Vision

I. INTRODUCTION

Autonomous vehicles are transforming the modern transportation industry through advancements in artificial intelligence and deep learning. Traditional autonomous systems depend on multiple independent modules such as lane detection, object recognition, localization, and path planning. Although effective, these systems require extensive manual feature engineering and complex integration techniques.

Behavioral cloning introduces an end-to-end learning approach where a neural network directly learns steering behavior from human driving examples. Instead of manually programming lane detection algorithms, the model automatically learns relevant road features directly from raw camera images.

The primary objective of this project is to develop a CNN-based autonomous steering prediction system capable of driving inside the Udacity Self-Driving Car Simulator environment.

II. WHY THIS PROJECT

This project was selected because autonomous driving is one of the fastest-growing applications of artificial intelligence and deep learning. The project combines multiple important computer science concepts including:

- Machine Learning

- Deep Learning
- Computer Vision
- Image Processing
- Real-Time Systems
- Artificial Intelligence

The project also provides practical implementation experience instead of only theoretical understanding.

Another important reason for selecting this project is that behavioral cloning provides a complete end-to-end deep learning workflow including:

- Dataset processing
- Data visualization
- Model training
- Performance optimization
- Real-time deployment

This makes the project highly valuable from both academic and practical perspectives.

III. WHY NOT OTHER PROJECTS

Many beginner machine learning projects such as simple image classification or digit recognition mainly focus on prediction accuracy only. Although useful, those projects do not provide real-time system implementation experience.

Compared to traditional beginner projects, autonomous driving introduces several advanced challenges including:

- Continuous regression prediction
- Real-time inference
- Image preprocessing
- Steering correction
- Dynamic throttle control
- Simulator deployment

This project was preferred because it combines theoretical knowledge with practical engineering implementation. It also demonstrates a real-world application of deep learning instead of only offline prediction tasks.

IV. LITERATURE REVIEW

A. Behavioral Cloning

Behavioral cloning is a supervised imitation learning technique where a model imitates human driving behavior using

demonstration data. The neural network learns a mapping between road images and steering angles.

Behavioral cloning is useful because it reduces manual feature engineering requirements and allows the model to automatically learn road features directly from image data.

However, behavioral cloning also has limitations. The model heavily depends on dataset quality and may fail in unseen driving conditions.

B. NVIDIA End-to-End Architecture

NVIDIA proposed one of the most influential autonomous driving CNN architectures. Their system demonstrated that convolutional neural networks can directly predict steering commands from front camera images without explicit lane detection.

The architecture inspired many modern end-to-end autonomous driving systems due to its simplicity and effectiveness.

V. THEORETICAL KNOWLEDGE

This project is strongly based on several theoretical concepts from machine learning and computer vision.

A. Convolutional Neural Networks

CNNs are specialized neural networks designed for image processing tasks. Instead of manually extracting image features, CNNs automatically learn visual patterns through convolution operations.

CNNs are effective because they preserve spatial relationships between neighboring pixels.

Early CNN layers learn simple features such as:

- Edges
- Curves
- Lines

Deeper layers learn complex features including:

- Road structures
- Lane patterns
- Steering-related visual information

B. Regression Learning

Steering angle prediction is a regression problem because steering values are continuous numerical values rather than discrete classes.

For this reason, Mean Squared Error (MSE) is used as the loss function.

$$MSE = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2 \quad (1)$$

VI. PRACTICAL KNOWLEDGE

This project provided significant practical implementation experience in deep learning engineering.

Several practical skills were developed during implementation including:

- Dataset handling
- Data preprocessing

- Image augmentation
- CNN model building
- GPU-based training
- Real-time deployment
- Model optimization

The project also demonstrated how theoretical machine learning concepts are applied in real-world systems.

Google Colab GPU acceleration was used to reduce training time and improve computational efficiency.

VII. EXPERIMENTAL KNOWLEDGE

Multiple experiments were performed during training and deployment.

A. Steering Imbalance Observation

Dataset exploration showed that most steering angles were close to zero because vehicles spend most time driving straight.

This caused steering imbalance where the model initially became biased toward straight predictions.

B. Augmentation Experiment

Several augmentation techniques were tested including:

- Horizontal flipping
- Translation
- Brightness adjustment

Experimental observations showed that augmentation significantly improved recovery behavior and reduced overfitting.

C. Epoch Experiment

Initially, larger epoch values caused overfitting.

After experimentation, 10 epochs produced better validation performance and more stable simulator driving behavior.

VIII. METHODOLOGY

A. Dataset

The project uses the Udacity Self-Driving Car Simulator dataset containing:

- Center camera images
- Left camera images
- Right camera images
- Steering angle labels
- Vehicle speed
- Brake and throttle information

The dataset is stored using the `driving_log.csv` file.

B. GPU Configuration

The model is trained using Google Colab with GPU acceleration.

Listing 1. GPU Availability Check

```
import tensorflow as tf

print('TensorFlow:', tf.__version__)
print('GPU available:',
      tf.config.list_physical_devices('GPU'))
```

C. Dataset Extraction

Listing 2. Dataset Download and Extraction

```
import os, zipfile, urllib.request

DATA_URL = 'https://d17h27t6h515a5.cloudfront.net/
topher/2016/December/584f6edd_data/data.zip'

ZIP_PATH = '/content/data.zip'
DATA_PATH = '/content/data/'
```

D. Dataset Exploration

Listing 3. Dataset Loading

```
import pandas as pd

cols = ['center', 'left', 'right',
        'steering', 'throttle',
        'brake', 'speed']

df = pd.read_csv(LOG_FILE,
                 names=cols,
                 header=0)
```

E. Image Preprocessing

Several preprocessing operations are applied before training:

- 1) Image cropping
- 2) Image resizing
- 3) YUV color conversion
- 4) Pixel normalization

$$I_{normalized} = \frac{I}{255.0} \quad (2)$$

F. Data Augmentation

To reduce overfitting and improve generalization, several augmentation techniques are applied:

- Random horizontal flipping
- Random translation
- Random brightness adjustment

$$\theta_{new} = -\theta \quad (3)$$

G. CNN Architecture

The project implements the NVIDIA CNN architecture consisting of:

- Convolutional layers
- ReLU activation functions
- Fully connected layers
- Dropout layers
- Regression output layer

H. Training Configuration

TABLE I
TRAINING CONFIGURATION

Parameter	Value
Optimizer	Adam
Loss Function	Mean Squared Error
Batch Size	32
Epochs	10
Learning Rate	0.0001
Early Stopping	Enabled
Model Checkpoint	Enabled

IX. IMPLEMENTATION

A. Development Environment

The project is developed using Google Colab with GPU acceleration.

Major libraries used include:

- TensorFlow
- Keras
- NumPy
- OpenCV
- Pandas
- Matplotlib
- SocketIO
- Eventlet

B. Real-Time Deployment

The trained model is deployed using the `drive.py` script. The deployment pipeline performs the following operations:

- 1) Receives telemetry data
- 2) Decodes simulator images
- 3) Applies preprocessing
- 4) Predicts steering angle
- 5) Calculates throttle
- 6) Sends control commands

$$Throttle = 1.0 - (|SteeringAngle| \times 2.0) \quad (4)$$

X. RESULTS AND DISCUSSION

The CNN model successfully converged during training using Mean Squared Error loss minimization.

The trained model demonstrated:

- Stable lane following
- Smooth turning capability
- Recovery from moderate lane displacement
- Real-time autonomous navigation

Experimental observations showed that preprocessing and augmentation significantly improved driving stability.

XI. CHALLENGES FACED

Several challenges were encountered during implementation:

- Overfitting
- Steering imbalance
- Sharp turn instability
- Computational complexity

These issues were mitigated using augmentation, dropout regularization, and early stopping techniques.

XII. CONCLUSION

This project successfully implemented an end-to-end behavioral cloning system for autonomous steering prediction using deep convolutional neural networks.

The project combined theoretical knowledge, practical implementation, and experimental evaluation in a real-time autonomous driving environment.

The model successfully learned steering behavior directly from raw camera images without handcrafted lane detection algorithms.

XIII. FUTURE WORK

Future improvements may include:

- Larger datasets
- Reinforcement learning integration
- LiDAR integration
- Real-world deployment
- Sensor fusion techniques

ACKNOWLEDGMENT

The authors acknowledge the use of TensorFlow, Keras, OpenCV, Google Colab, and the Udacity Self-Driving Car Simulator dataset during the development of this project.

REFERENCES

- [1] M. Bojarski et al., "End to End Learning for Self-Driving Cars," NVIDIA, 2016.
- [2] TensorFlow Documentation. Available: <https://www.tensorflow.org/>
- [3] Keras Documentation. Available: <https://keras.io/>
- [4] OpenCV Documentation. Available: <https://opencv.org/>
- [5] NumPy Documentation. Available: <https://numpy.org/>
- [6] Python Documentation. Available: <https://www.python.org/>
- [7] Google Colab Documentation. Available: <https://colab.research.google.com/>